

UZUPEŁNIENIE MODUŁU 03

PySpark i Spark SQL

Oba interfejsy obok siebie: odczyt, transformacje, agregacje

Spis treści

SEKCJA 1	Odczyt plików CSV	3
SEKCJA 2	Odczyt plików JSON	5
SEKCJA 3	Odczyt plików Parquet	6
SEKCJA 4	Eksploracja danych	6
SEKCJA 5	Przekształcanie kolumn	7
SEKCJA 6	Usuwanie duplikatów i sortowanie	9
SEKCJA 7	Logika warunkowa	9
SEKCJA 8	Filtrowanie wierszy	10
SEKCJA 9	Agregacja	13
SEKCJA 10	Widoki tymczasowe i SQL	15
SEKCJA 11	Zapis do Delta	16
SEKCJA 12	Ściągawka	17

Ten materiał pokazuje każdą operację z modułu 03 w obu interfejsach używanych na szkoleniu: PySpark DataFrame API i Spark SQL. Oba są kompilowane przez optymalizator Catalyst do tego samego planu wykonania, więc wybór między nimi to kwestia czytelności i preferencji zespołu, a nie wydajności.

Przykłady korzystają ze zbiorów danych ze szkolenia: `customers` (10 000 wierszy), `orders` (100 000 wierszy) i `products` (2000 wierszy). Ścieżki wolumenów skrócono do `/Volumes/.../`.

SEKCJA 1

Odczyt plików CSV

Automatyczne wykrywanie schematu

PYSPARK

```
# inferSchema makes Spark read the file twice
customers_df = (
    spark.read
    .format("csv")
    .option("header", "true")
    .option("inferSchema", "true")
    .load("/Volumes/.../customers.csv")
)
```

SPARK SQL

```
-- read_files() infers the schema
SELECT *
FROM read_files(
    '/Volumes/.../customers.csv',
    format => 'csv',
    header => true
);
```

Jawna definicja schematu

PYSPARK

```
from pyspark.sql.types import StructType, StructField, StringType, DateType

schema = StructType([
    StructField("customer_id", StringType(), False),
    StructField("first_name", StringType(), True),
    StructField("email", StringType(), True),
    StructField("registration_date", DateType(), True),
])

customers_df = (
    spark.read
    .format("csv")
    .option("header", "true")
    .schema(schema)
    .load("/Volumes/.../customers.csv")
)
```

SPARK SQL

```
-- The schema is passed as a SQL string
SELECT *
FROM read_files(
    '/Volumes/.../customers.csv',
    format => 'csv',
    header => true,
    schema => 'customer_id STRING NOT NULL,
              first_name STRING,
              email STRING,
              registration_date DATE'
);
```

Utworzenie tabeli Bronze

PYSPARK

```
(customers_df.write
    .format("delta")
    .mode("overwrite")
    .saveAsTable("bronze.customers"))
```

SPARK SQL

```
CREATE OR REPLACE TABLE bronze.customers AS
SELECT
    *,
    _metadata.file_path AS source_file
FROM read_files(
    '/Volumes/.../customers.csv',
    format => 'csv',
    header => true
);
```

SEKCJA 2

Odczyt plików JSON

Automatyczne wykrywanie schematu

PYSPARK

```
orders_df = (  
    spark.read  
    .format("json")  
    .load("/Volumes/.../orders.json")  
)
```

SPARK SQL

```
SELECT *  
FROM read_files(  
    '/Volumes/.../orders.json',  
    format => 'json'  
);
```

Odczyt wybranych kolumn

PYSPARK

```
from pyspark.sql.types import DoubleType  
  
orders_schema = StructType([  
    StructField("order_id", StringType(), True),  
    StructField("customer_id", StringType(), False),  
    StructField("total_amount", DoubleType(), False),  
    StructField("payment_method", StringType(), False),  
)  
  
orders_df = (  
    spark.read  
    .format("json")  
    .schema(orders_schema)  
    .load("/Volumes/.../orders.json")  
)
```

SPARK SQL

```
SELECT  
    order_id,  
    customer_id,  
    total_amount,  
    payment_method  
FROM read_files(  
    '/Volumes/.../orders.json',  
    format => 'json'  
);
```

SEKCJA 3

Odczyt plików Parquet

Pliki Parquet przechowują własny schemat, więc nie są potrzebne ani `inferSchema`, ani `StructType`. Parquet jest też formatem plików, na którym opiera się Delta Lake, a jego kolumnowy układ pozwala Sparkowi pomijać niepotrzebne dane.

PYSPARK

```
products_df = (\n    spark.read\n      .format("parquet")\n      .load("/Volumes/.../products.parquet")\n)
```

SPARK SQL

```
SELECT *\nFROM read_files(\n  '/Volumes/.../products.parquet',\n  format => 'parquet'\n);
```

SEKCJA 4

Eksploracja danych

Operacja	PySpark	Spark SQL
Podgląd wierszy	<code>df.show(5)</code> lub <code>display(df.limit(5))</code>	<code>SELECT * FROM t LIMIT 5</code>
Schemat	<code>df.printSchema()</code>	<code>DESCRIBE t</code>
Statystyki opisowe	<code>df.describe()</code> , <code>df.summary()</code>	<code>SELECT MIN(x), MAX(x), AVG(x), COUNT(*) FROM t</code>
Liczba wierszy	<code>df.count()</code>	<code>SELECT COUNT(*) FROM t</code>
Unikalne wartości	<code>df.select("col").distinct()</code>	<code>SELECT DISTINCT col FROM t</code>

SEKCJA 5

Przekształcanie kolumn

Wybór kolumn i wyrażenia

PYSPARK

```
from pyspark.sql.functions import col, upper

df.select("customer_id", "first_name", "email")

df.select(
    col("customer_id"),
    upper(col("first_name")).alias("first_name_upper"),
    col("email"),
)
```

SPARK SQL

```
SELECT customer_id, first_name, email
FROM customers;

SELECT
    customer_id,
    UPPER(first_name) AS first_name_upper,
    email
FROM customers;
```

Dodanie lub zmiana kolumny

PYSPARK

```
from pyspark.sql.functions import concat, lit, year

df.withColumn("full_name", concat(col("first_name"), lit(" "), col("last_name")))

df.withColumn("reg_year", year(col("registration_date")))
```

SPARK SQL

```
SELECT *, CONCAT(first_name, ' ', last_name) AS full_name
FROM customers;

SELECT *, YEAR(registration_date) AS reg_year
FROM customers;
```

Zmiana typu

TRY_CAST zwraca null zamiast zgłaszać błąd, gdy wartości nie da się przekonwertować.

PYSPARK

```
df.withColumn("reg_date", col("registration_date").cast(DateType()))

df.withColumn("id_int", col("customer_id").cast("int"))
```

SPARK SQL

```
SELECT CAST(registration_date AS DATE) AS reg_date
FROM customers;

SELECT TRY_CAST(customer_id AS INT) AS id_int
FROM customers;
```

Zmiana nazw kolumn

Przy zmianie nazw kilku kolumn jeden select z aliasami jest czytelniejszy niż ciąg wywołań withColumnRenamed.

PYSPARK

```
df.withColumnRenamed("customer_id", "id")

df.select(
    col("customer_id").alias("id"),
    col("first_name").alias("fname"),
    col("last_name").alias("lname"),
)
```

SPARK SQL

```
SELECT
    customer_id AS id,
    first_name  AS fname,
    last_name   AS lname
FROM customers;
```

Usuwanie kolumn

SELECT * EXCEPT (...) to składnia Databricks SQL.

PYSPARK

```
df.drop("email")
df.drop("email", "city", "country")
```

SPARK SQL

```
SELECT * EXCEPT (email, city, country)
FROM customers;
```

SEKCJA 6

Usuwanie duplikatów i sortowanie

Unikalne wiersze

PYSPARK

```
df.select("country").distinct()

df.select("city", "country").dropDuplicates(["city", "country"])
```

SPARK SQL

```
SELECT DISTINCT country
FROM customers;

SELECT DISTINCT city, country
FROM customers;
```

Sortowanie

PYSPARK

```
from pyspark.sql.functions import asc, desc

df.orderBy("registration_date")

df.orderBy(asc("country"), desc("registration_date"))
```

SPARK SQL

```
SELECT * FROM customers
ORDER BY registration_date;

SELECT * FROM customers
ORDER BY country ASC, registration_date DESC;
```

SEKCJA 7

Logika warunkowa

W PySpark łączy się kolejne wywołania `when()` i kończy je `otherwise()`. W SQL tę samą logikę zapisuje się jako `CASE WHEN ... THEN ... ELSE ... END`, a wartości null zastępują `COALESCE`, `IFNULL` lub `NVL`.

PYSPARK

```
from pyspark.sql.functions import when

df.withColumn(
    "customer_tier",
    when(year(col("registration_date")) >= 2023, "New")
    .when(year(col("registration_date")) >= 2021, "Regular")
    .otherwise("Loyal"),
)

df.withColumn(
    "city_safe",
    when(col("city").isNull(), lit("Unknown")).otherwise(col("city")),
)
```

SPARK SQL

```
SELECT *,
    CASE
        WHEN YEAR(registration_date) >= 2023 THEN 'New'
        WHEN YEAR(registration_date) >= 2021 THEN 'Regular'
        ELSE 'Loyal'
    END AS customer_tier
FROM customers;

SELECT *, COALESCE(city, 'Unknown') AS city_safe
FROM customers;
```

SEKCJA 8

Filtrowanie wierszy

Proste warunki

W PySpark `filter()` i `where()` działają identycznie.

PYSPARK

```
df.filter(col("country") == "USA")
df.where(col("city") == "New York")

df.filter("country = 'USA'")      # a SQL expression as a string
```

SPARK SQL

```
SELECT * FROM customers WHERE country = 'USA';

SELECT * FROM customers WHERE city = 'New York';
```

Łączenie warunków

W PySpark każdy warunek łączony operatorem `&` (i) lub `|` (lub) musi być ujęty w nawiasy.

PYSPARK

```
df.filter(
    (col("country") == "USA") &
    (year(col("registration_date")) == 2023)
)

df.filter(
    (col("country") == "USA") |
    (col("country") == "UK")
)
```

SPARK SQL

```
SELECT * FROM customers
WHERE country = 'USA'
  AND YEAR(registration_date) = 2023;

SELECT * FROM customers
WHERE country = 'USA'
  OR country = 'UK';
```

Filtrowanie według listy wartości

PYSPARK

```
countries = ["USA", "UK", "Germany", "France"]

df.filter(col("country").isin(countries))
df.filter(~col("country").isin(countries))
```

SPARK SQL

```
SELECT * FROM customers
WHERE country IN ('USA', 'UK', 'Germany', 'France');

SELECT * FROM customers
WHERE country NOT IN ('USA', 'UK', 'Germany', 'France');
```

Wartości null

W PySpark porównanie `== None` nie znajduje wartości null; używaj `isNull()` lub `isNotNull()`.

PYSPARK

```
df.filter(col("email").isNotNull())
df.filter(col("city").isNull())
```

SPARK SQL

```
SELECT * FROM customers WHERE email IS NOT NULL;

SELECT * FROM customers WHERE city IS NULL;
```

Uzupełnianie lub usuwanie brakujących wartości

PYSPARK

```
df.fillna("Unknown", subset=["city"])
df.fillna({"city": "Unknown", "country": "N/A"})

df.dropna() # rows with any null
df.dropna(how="all") # rows where every value is null
df.dropna(subset=["customer_id"]) # rows with a null in selected columns
```

SPARK SQL

```
SELECT COALESCE(city, 'Unknown') AS city,
       IFNULL(country, 'N/A') AS country
FROM customers;

SELECT * FROM customers
WHERE customer_id IS NOT NULL;
```

Dopasowywanie tekstu

PYSPARK

```
df.filter(col("email").like("%@gmail.com"))
df.filter(col("city").contains("New"))
df.filter(col("first_name").startswith("J"))
df.filter(col("email").endswith(".com"))
df.filter(col("email").rlike(r"^[a-z]+\.[a-z]+"))
```

SPARK SQL

```
SELECT * FROM customers WHERE email LIKE '%@gmail.com';
SELECT * FROM customers WHERE CONTAINS(city, 'New');
SELECT * FROM customers WHERE STARTSWITH(first_name, 'J');
SELECT * FROM customers WHERE ENDSWITH(email, '.com');
SELECT * FROM customers WHERE REGEXP_LIKE(email, '^[a-z]+\.[a-z]+');
```

SEKCJA 9

Agregacja

Liczności, sumy i średnie

PYSPARK

```
from pyspark.sql.functions import count, sum, avg

df.groupBy("country").count().orderBy(desc("count"))

(orders_df.groupBy("payment_method")
 .agg(sum("total_amount").alias("revenue"),
      avg("total_amount").alias("avg_order"),
      count("*").alias("n_orders"))
 .orderBy(desc("revenue")))
```

SPARK SQL

```
SELECT country, COUNT(*) AS count
FROM customers
GROUP BY country
ORDER BY count DESC;

SELECT
    payment_method,
    SUM(total_amount) AS revenue,
    AVG(total_amount) AS avg_order,
    COUNT(*)          AS n_orders
FROM orders
GROUP BY payment_method
ORDER BY revenue DESC;
```

Minimum i maksimum

PYSPARK

```
from pyspark.sql.functions import min, max

orders_df.agg(
    min("total_amount").alias("min_amount"),
    max("total_amount").alias("max_amount"),
    avg("total_amount").alias("avg_amount"),
)
```

SPARK SQL

```
SELECT
    MIN(total_amount) AS min_amount,
    MAX(total_amount) AS max_amount,
    AVG(total_amount) AS avg_amount
FROM orders;
```

Kilka agregatów w jednym przebiegu

Obliczenie wszystkich agregatów w jednym wywołaniu `agg()` odczytuje dane tylko raz.

PYSPARK

```
(orders_df.groupBy("customer_id")
  .agg(count("*").alias("total_orders"),
    sum("total_amount").alias("total_spent"),
    avg("total_amount").alias("avg_order"),
    min("total_amount").alias("min_order"),
    max("total_amount").alias("max_order"))
  .orderBy(desc("total_spent")))
```

SPARK SQL

```
SELECT
  customer_id,
  COUNT(*)      AS total_orders,
  SUM(total_amount) AS total_spent,
  AVG(total_amount) AS avg_order,
  MIN(total_amount) AS min_order,
  MAX(total_amount) AS max_order
FROM orders
GROUP BY customer_id
ORDER BY total_spent DESC;
```

Filtrowanie po agregacji

`filter()` przed `groupBy()` odpowiada `WHERE` w SQL, a `filter()` po `groupBy().agg()` odpowiada `HAVING`.

PYSPARK

```
(orders_df.groupBy("customer_id")
  .agg(count("*").alias("order_count"),
    sum("total_amount").alias("total_spent"))
  .filter(col("order_count") > 5)
  .orderBy(desc("order_count")))
```

SPARK SQL

```
SELECT
  customer_id,
  COUNT(*)      AS order_count,
  SUM(total_amount) AS total_spent
FROM orders
GROUP BY customer_id
HAVING COUNT(*) > 5
ORDER BY order_count DESC;
```

SEKCJA 10

Widoki tymczasowe i SQL

Typ widoku	Widoczny dla	PySpark	Spark SQL	Czas życia
Widok tymczasowy	Bieżącej sesji	<code>df.createOrReplaceTempView("v")</code>	<code>CREATE OR REPLACE TEMP VIEW v AS ...</code>	Do końca sesji
Globalny widok tymczasowy	Wszystkich sesji na klastrze	<code>df.createOrReplaceGlobalTempView("v")</code>	Odpytywany jako <code>global_temp.v</code>	Do restartu klastra
Widok w Unity Catalog	Każdego z uprawnieniami	<code>spark.sql("CREATE VIEW ...")</code>	<code>CREATE VIEW catalog.schema.v AS ...</code>	Trwały

Rejestracja i odpytanie widoku

PYSPARK

```
customers_df.createOrReplaceTempView("customers")
orders_df.createOrReplaceTempView("orders")

result = spark.sql("""
    SELECT customer_id, COUNT(*) AS order_count
    FROM orders
    GROUP BY customer_id
    ORDER BY order_count DESC
    LIMIT 10
""")
display(result)
```

SPARK SQL

```
CREATE OR REPLACE TEMP VIEW orders AS
SELECT *
FROM read_files('/Volumes/.../orders.json', format => 'json');

SELECT customer_id, COUNT(*) AS order_count
FROM orders
GROUP BY customer_id
ORDER BY order_count DESC
LIMIT 10;
```

Join w SQL

To samo zapytanie można uruchomić z Pythona przez `spark.sql()` albo bezpośrednio w komórce `%sql`.

SPARK SQL

```
SELECT
  c.customer_id,
  c.first_name,
  c.country,
  COUNT(o.order_id) AS total_orders,
  SUM(o.total_amount) AS total_spent,
  AVG(o.total_amount) AS avg_order
FROM customers c
LEFT JOIN orders o
  ON c.customer_id = TRY_CAST(o.customer_id AS STRING)
GROUP BY c.customer_id, c.first_name, c.country
HAVING COUNT(o.order_id) > 0
ORDER BY total_spent DESC
LIMIT 10;
```

SEKCJA 11

Zapis do Delta

Tryby zapisu w PySpark to `overwrite`, `append`, `ignore` i `error`.

PYSPARK

```
(df.write
  .format("delta")
  .mode("overwrite")
  .saveAsTable("silver.customers_clean"))

(df.write
  .format("delta")
  .mode("append")
  .saveAsTable("silver.customers_clean"))
```

SPARK SQL

```
CREATE OR REPLACE TABLE silver.customers_clean AS
SELECT * FROM customers
WHERE customer_id IS NOT NULL;

INSERT INTO silver.customers_clean
SELECT * FROM customers_new;
```

SEKCJA 12

Ściągawka

Operacja	PySpark	Spark SQL
Odczyt CSV	<code>spark.read.format("csv").option("header", "true").load(path)</code>	<code>read_files(path, format => 'csv', header => true)</code>
Odczyt JSON	<code>spark.read.format("json").load(path)</code>	<code>read_files(path, format => 'json')</code>
Odczyt Parquet	<code>spark.read.format("parquet").load(path)</code>	<code>read_files(path, format => 'parquet')</code>
Wykrycie schematu	<code>.option("inferSchema", "true")</code>	Automatycznie w <code>read_files()</code>
Wybór kolumn	<code>df.select("c1", "c2")</code>	<code>SELECT c1, c2 FROM t</code>
Nowa kolumna	<code>df.withColumn("new", expr)</code>	<code>SELECT *, expr AS new FROM t</code>
Zmiana nazwy kolumny	<code>df.withColumnRenamed("old", "new")</code>	<code>SELECT old AS new FROM t</code>
Usunięcie kolumn	<code>df.drop("c1", "c2")</code>	<code>SELECT * EXCEPT (c1, c2) FROM t</code>
Zmiana typu	<code>col("x").cast("int")</code>	<code>CAST(x AS INT), TRY_CAST(x AS INT)</code>
Wartość warunkowa	<code>when(cond, v1).otherwise(v2)</code>	<code>CASE WHEN cond THEN v1 ELSE v2 END</code>
Zastąpienie null	<code>df.fillna("val", subset=["col"])</code>	<code>COALESCE(col, 'val')</code>
Filtr	<code>df.filter(col("x") == "val")</code>	<code>WHERE x = 'val'</code>
I, lub	<code>(c1) & (c2), (c1) (c2)</code>	<code>WHERE c1 AND c2, WHERE c1 OR c2</code>
Lista wartości	<code>col("x").isin(["a", "b"])</code>	<code>WHERE x IN ('a', 'b')</code>
Test null	<code>col("x").isNull(), .isNotNull()</code>	<code>WHERE x IS NULL, IS NOT NULL</code>
Wzorec	<code>col("x").like("%pat%")</code>	<code>WHERE x LIKE '%pat%'</code>
Wyrażenie regularne	<code>col("x").rlike(r"^[a-z]+")</code>	<code>WHERE REGEXP_LIKE(x, '^[a-z]+')</code>
Unikalne wiersze	<code>df.distinct(), df.dropDuplicates(["col"])</code>	<code>SELECT DISTINCT ...</code>
Sortowanie	<code>df.orderBy(asc("x"), desc("y"))</code>	<code>ORDER BY x ASC, y DESC</code>
Grupowanie	<code>df.groupBy("col").agg(...)</code>	<code>GROUP BY col</code>
Agregaty	<code>count("*"), sum("c"), avg("c"), min("c"), max("c")</code>	<code>COUNT(*), SUM(c), AVG(c), MIN(c), MAX(c)</code>
Filtr grup	<code>.filter() po .groupBy().agg()</code>	<code>HAVING COUNT(*) > n</code>
Widok tymczasowy	<code>df.createOrReplaceTempView("v")</code>	<code>CREATE OR REPLACE TEMP VIEW v AS ...</code>
SQL z Pythona	<code>spark.sql("SELECT ...")</code>	Komórka <code>%sql</code>
Nadpisanie tabeli	<code>.write.mode("overwrite").saveAsTable("t")</code>	<code>CREATE OR REPLACE TABLE t AS SELECT ...</code>
Dopisanie do tabeli	<code>.write.mode("append").saveAsTable("t")</code>	<code>INSERT INTO t SELECT ...</code>